

There are **10 ways** your LLM app can be attacked.

Most engineers know two.

A visual map of the complete LLM threat landscape.
One threat per slide. How it works. Why it matters.

LLM01 Prompt Injection

LLM02 Sensitive Information Disclosure

LLM03 Supply Chain Vulnerabilities

LLM04 Data and Model Poisoning

LLM05 Improper Output Handling

LLM06 Excessive Agency

LLM07 System Prompt Leakage

LLM08-10 Three more threats

What is OWASP

Top 10 for LLMs?

OWASP is the Open Worldwide Application Security Project. For two decades their Top 10 list has been the industry standard for web application security. In 2023 they published a version built specifically for LLM applications, grounded in real vulnerabilities found in real production systems.

WHAT IT IS

Industry standard

Community-maintained

First published 2023

Updated 2025

WHY IT MATTERS

Built from real incidents

Production-proven threats

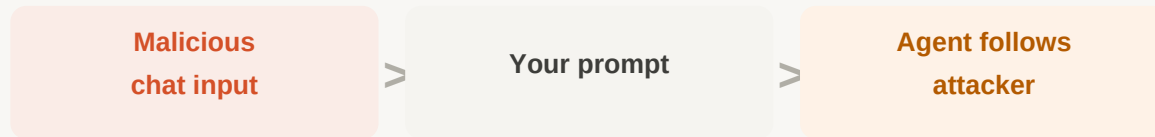
Shared security language

Used in audits and reviews

Most engineers building with LLMs today have never read it.

The attacker's instructions, not yours.

DIRECT INJECTION



INDIRECT INJECTION -- covered in depth in Post 2



Ranked first because it enables or amplifies every other threat on this list.

Direct injection comes through the chat interface. Indirect injection weaponises data your system retrieves and trusts: documents, emails, webpages. The user types nothing harmful. The data does it for them.

The model reveals what it should protect.

● Training data memorisation

The model learned from data containing PII, credentials, or proprietary content. Under the right prompt it reproduces that content verbatim.

● System prompt extraction

An attacker asks the model to repeat, translate, or reformat its own instructions. Your business logic and constraints become visible.

● Cross-session data leakage

Context is not properly isolated between users. Data from one session surfaces in another's silently, with no error or alert.

The threat enters before you write a line of code.

Pre-training data

poisoned at source before the model exists

V

Fine-tuning dataset

corrupted before you download or apply it

V

Third-party plugin

malicious functionality inside a tool integration

V

Your application

inherits every problem above at inference time

You did not build the model. You did not control what it learned from. You are responsible for what it does in your system.

Attack at training time.

Damage at inference time.

WHEN IT HAPPENS

Training or fine-tuning phase. The attacker contaminates data before the model ever sees production.

WHEN YOU SEE IT

Inference phase. Weeks or months later.

Normal behaviour 99% of the time.

Triggered only under specific conditions the attacker designed.

Training time attack

Gap (months)

Inference time discovery

The gap between attack and discovery is what makes poisoning uniquely dangerous. By the time it surfaces, it has often been in production for months.

Classic injection. New entry point.

The model generates output. Your system uses that output to do something downstream: query a database, render HTML, execute a shell command, call an API. If the output is not validated before it reaches those systems, it becomes an injection vector.

LLM generates:

SELECT * FROM users WHERE role='admin' --

**Database runs it
unsanitised**

SQL injection

**Frontend renders
it directly**

XSS attack

**Agent executes
as a command**

Shell injection

SQL injection and XSS are decades old. What is new is that the injection now originates from a model that generates malformed output with complete confidence.

More access than the task requires.

WHAT IT NEEDED

Read files

Query database

Send one email

WHAT IT WAS GIVEN

Read, write, delete files

Full database access

Full email and calendar

CRM and file system

When the agent is manipulated, the blast radius is determined entirely by its permissions. Excessive agency is a multiplier on every other threat.

Phase 3 of this series covers agent access control in full.

The principle: an agent should have the minimum access required to complete its task. No more.

Your instructions. Their intelligence.

YOUR SYSTEM PROMPT CONTAINS

Business logic

Persona rules

Constraints

Tool access

Guard conditions

How attackers extract it:

- Ask the model to repeat its instructions directly
- Ask it to translate or summarise 'everything it knows'
- Use a roleplay scenario where revealing is in-character
- Trigger self-disclosure via indirect prompt injection

The model is not reliably capable of keeping a secret. Design your system with that assumption built in.

Each one gets its own post.

● Vector and Embedding Attacks

Poisoning the embedding space so that a RAG pipeline retrieves malicious content even from clean-looking queries. The attack is invisible at the document level and lives inside the vector representation itself.

● Misinformation

The model generates confident, fluent, incorrect output. Downstream systems or users act on it. The damage is not a crash. It is a decision made on bad information, with no error log to trace.

● Unbounded Consumption

No rate limiting. No token budgets. No cost controls. An attacker exhausts your inference budget or degrades the service for every legitimate user simultaneously.

Every one of these exploits the same gap.

Prompt injection

Trusted instructions mixed with untrusted data

Info disclosure

Trusted retrieval of information that should be protected

Supply chain

Trusted base model with unverified provenance

Output handling

Trusted model output used without validation

**Trust nothing by default.
Verify at every boundary.**

This was the map.

The next 8 weeks are the territory.

We go deep on each category. One at a time. With the same level of precision as the prompt injection post.

Next post: how input validation in an AI application differs from every pattern you learned before LLMs existed.

**Which of these 10 have you encountered
in a system you built?**